Model Card — CodeVision AI Analysis Engine

Multi-Pass Codebase Intelligence System · v1.0

Claude Sonnet | Tree-sitter AST | 4-Pass Pipeline | Business Mapper | TypeScript

4	44	400	~45s
AI PASSES	TECH SIGS	MAX FILES	P95 LATENCY

1 Model Overview

What This Model Does

The CodeVision AI Analysis Engine is a **multi-pass LLM-powered pipeline** that transforms raw GitHub repositories into structured, business-friendly architecture intelligence. It combines **deterministic code analysis** (AST parsing, dependency graphs, pattern detection) with **progressive AI reasoning** across 4 sequential passes to produce architecture diagrams, business capability maps, risk assessments, and executive narratives.

Unlike traditional static analysis tools that only detect code-level patterns, this system bridges the gap between **technical implementation and business understanding**, making codebases accessible to founders, product managers, and investors who need to evaluate software architecture without reading code.

🧠

Core LLM — Claude Sonnet

Anthropic Claude Sonnet as reasoning backbone. Used for code understanding, architecture inference, business context generation, and interactive Q&A. Not fine-tuned — uses carefully engineered system prompts with structured JSON output schemas.

🌳

AST Parser — Tree-sitter

Tree-sitter and ast-grep for deterministic structural analysis. Parses source code into abstract syntax trees to extract classes, functions, imports, and exports. Builds dependency graphs and detects architecture patterns (MVC, Clean Architecture, microservices).

🔍

Business Mapper

Keyword-based capability mapping engine that translates technical architecture nodes into 8 business capabilities across 3 strategic domains. Uses confidence scoring (0.45–0.95) and generates deterministic user journey graphs as fallback when AI output is incomplete.

Model Processing Pipeline

📥 **Raw Code Input**
GitHub repo URL. Up to 400 prioritized files, 16+ languages

→

🌳 **AST Parsing**
Tree-sitter for 5 languages; regex fallback for the rest

→

⚙️ **Deterministic Signals**
File scoring, pattern detection, 44 tech fingerprints

→

🤖 **Claude AI Analysis**
4 passes: Summaries → Arch Graph → Business → Narrative

→

🗺️ **Business Mapping**
8 capabilities, 3 strategic domains, user journey graphs

→

📊 **Structured Output**
6 UI-ready views: arch, risk, tech stack, capabilities, Q&A

2 Intended Use

The model serves as the **core intelligence engine** within CodeVision, automating architecture analysis that would traditionally require a senior engineer spending hours or days reviewing a codebase.

Non-Technical Founders

Understand what their engineering team has built, evaluate technical decisions, and communicate architecture to investors.

"What does my codebase actually do and how risky is the architecture?"

Product Managers

Map technical systems to business capabilities, identify user journey gaps, and assess feature delivery risk.

"Which business capabilities are well-supported vs. underdeveloped?"

Technical Due Diligence

Rapidly assess technology stack, architecture patterns, risk factors, and technical debt before investment or acquisition.

"What are the top risks in this codebase and what would remediation cost?"

Engineering Teams

Onboard onto new codebases, understand dependency structures, and track architecture evolution across versions.

"How does data flow from API to database? What changed between v1 and v2?"

Out of Scope

— Automated code generation or modification
— Runtime performance profiling or load testing
— Replacing human code review for merge decisions
— Security vulnerability scanning (SAST/DAST)
— Compliance auditing (SOC2, HIPAA, GDPR)

---

3 Data Inputs & Characteristics

The model operates on two primary data sources at inference time: source code from a GitHub repository and optional requirement documents uploaded by the user. There is no separate training dataset — the system leverages Claude Sonnet's pre-trained knowledge combined with deterministic code analysis signals.

📦 Source Code (Primary)

SOURCE	GitHub repos (public or private with token)
LANGUAGES	16+ (Python, JS, TS, Go, Java, Ruby, Rust, C/C++, PHP, Swift, Kotlin...)
SELECTION	Top 400 non-test files by priority score
SCORING	Entry points (5), Config (4), Routes (3), Models (3) + import count + size
EXCLUSIONS	node_modules, .git, dist, build, migrations, lock files, binaries
PARSING	Tree-sitter AST (5 langs: Python, JS, TS, Go, Java) + regex fallback

📄 Requirement Documents (Optional)

FORMATS	PDF, Markdown, Text, PNG, JPEG, GIF, WebP
MAX SIZE	10 MB per file
STORAGE	Supabase file storage with project scoping
PARSING	pdf-parse for PDFs, UTF-8 for text, Base64 for images
USAGE	Gap analysis — comparing implementation vs. requirements
FALLBACK	GitHub README auto-import if no docs uploaded

GitHub Metadata Signals

- ⭐ Stars
- 📝 Languages
- 👥 Contributors
- 📅 Commit History
- 🔀 Branch / Hash

Known Data Limitations

1. File selection capped at 400 — very large monorepos may have important files excluded from analysis
2. Tree-sitter AST parsing only covers 5 languages natively; others fall back to regex with lower fidelity
3. Binary files, minified code, and auto-generated files may be included, potentially skewing results
4. Private repositories require GitHub tokens; token permissions may limit metadata access
5. Requirement documents are optional — without them, gap analysis is not possible

---

4 Evaluation

Formal benchmarking is in progress. The metrics below reflect a mix of **directly measurable system signals** (latency, completion rate, parse success) and **qualitative estimates from informal spot-checks** on a small set of internal repositories. Numbers should be treated as directional until the golden-repo benchmark suite (see §6) is complete.

Technology Detection Precision — ~90%

Precision of 44-technology signature matching engine. Hardcoded fingerprints for frameworks, ORMs, and infra. Measures false-positive rate.

Measured: deterministic, verifiable against ground-truth package files.

Parse Success Rate — ~99%

Percentage of selected files successfully parsed (Tree-sitter or regex fallback). Failures are typically encoding issues or truly binary files.

Measured: logged per-job.

Analysis Completion Rate — ~88%

Jobs completing full 4-pass pipeline without falling back to single-pass. Primary failure cause: Claude API rate limits.

Measured: job status tracking in Supabase.

Architecture Accuracy — est. ~80%

Estimated % of generated architecture nodes corresponding to real, meaningful components. Based on informal spot-checks; not yet a formal benchmark.

Estimated: pending golden-repo suite (see §6).

Risk Signal Coverage — est. ~65%

Estimated % of common risk categories (missing tests, secrets in code, high complexity, no auth) detected by combined static + LLM analysis.

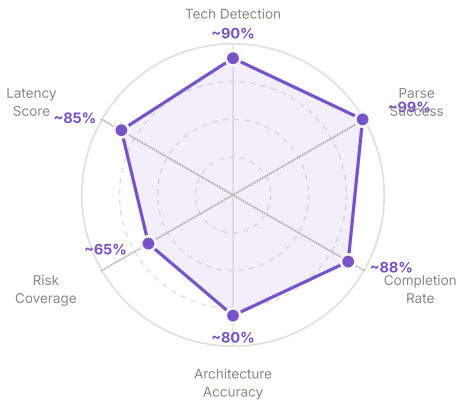
Estimated: informal audit of 5 repos.

Response Latency (P95) — ~45s

95th-percentile end-to-end time for a ~200-file repo including all 4 Claude passes + view generation. Target ceiling: 120s.

Measured: timed on internal test repos.

Performance Dimensions (estimated)



Why These Metrics Matter

Tech Detection — Signature-based detection is the most reliable layer. False positives here directly mislead the architecture graph and risk assessment.

Parse Success — Near-100% file coverage is required for the file selection heuristics to pick the correct entry points. Parsing failures cascade into incomplete dependency graphs.

Completion Rate — A failed multi-pass run degrades to single-pass, which produces a noticeably lower-quality output. Tracking this surfaces API reliability issues early.

Architecture Accuracy — The primary value metric. Incorrect node mapping directly misleads stakeholders; even a few phantom nodes erode trust in the full analysis.

Risk Coverage — Missing a critical risk pattern in a due-diligence context can have real financial consequences for the user.

Latency — User tolerance for wait times: <60s feels responsive, 60–120s acceptable with a progress bar, >120s risks abandonment.

Evaluation Methodology (In Progress)

We are building a golden-repo benchmark suite of 20+ open-source repositories with manually annotated ground-truth architecture maps. For each repo, a senior engineer produces the ground truth and we measure node/edge F1, technology detection F1, and risk recall. Latency and completion rate are already logged in production. Formal results are expected before the next product iteration.

5 Performance & Limitations

Where It Excels

Standard Web Applications
React/Next.js, Django/FastAPI, Express/Node.js apps with clear frontend-backend-database tiers produce reliable architecture maps.

Common Architecture Patterns
MVC, MVVM, Clean Architecture, REST APIs, and monolithic structures are reliably detected via both AST and LLM reasoning.

Technology Stack Identification
44-signature fingerprint library covers the most common web frameworks and infra tools with high precision.

Business Translation
Strong at converting technical structures into plain-English narratives for non-technical stakeholders via Claude's reasoning.

Small-to-Medium Repos
Best results on 50–500 source files. The 400-file selection window covers most of the codebase for repos this size.

Where It Struggles

Very Large Monorepos
Repos with 1000+ files may have critical modules excluded from the 400-file window, leading to incomplete architecture maps.

Uncommon Languages
Languages without Tree-sitter support (Elixir, Haskell, Scala) rely on regex parsing, missing nested structures and complex imports.

Microservice Architectures
Each microservice in a separate repo is analyzed independently — cross-service communication patterns are not visible.

Heavy Metaprogramming
Dynamic imports, reflection, and runtime module loading are invisible to static analysis — a fundamental limitation.

AI Hallucination Risk
LLM may infer connections or capabilities that don't exist, especially for unfamiliar or domain-specific frameworks.

Known Failure Modes

High **Rate Limit Exhaustion**

Multiple concurrent analyses can exhaust Claude API rate limits, causing passes 2–4 to fail mid-job.

Mitigation: Exponential backoff (3 retries), automatic single-pass fallback, and job queuing.

Medium **Token Overflow**

Very complex modules with deeply nested code can exceed the 80K token-per-pass limit, requiring truncation.

Mitigation: File prioritization and chunking limits total context; fallback to single-pass on overflow.

Low **Circular Dependency Tangles**

Heavily circular import graphs confuse topological sorting for layout, producing visually tangled diagrams.

Mitigation: Force-directed graph layout with cycle-breaking heuristics.

Low **Stale Cached Results**

Analysis results are cached for 24h. Rapid code changes may display an outdated architecture map.

Mitigation: Manual re-analysis trigger; version diff view surfaces changes between cached snapshots.

Low **Empty / Stub Repositories**

Repos with very few files or only scaffolding produce minimal, low-confidence analysis outputs.

Mitigation: Quality report flags low-evidence analyses with explicit confidence warnings.

Bias Considerations

The system may exhibit bias toward popular frameworks well-represented in Claude's training data. Niche technologies, proprietary frameworks, or domain-specific patterns (embedded systems, HPC, game engines) may be under-analyzed or misclassified. The 44-technology fingerprint library reflects a web-development-centric view. Business capability mapping uses keyword heuristics that favor SaaS business models.

6 Improvement Path

Implemented Improvements

Multi-Pass Pipeline (from Single-Pass)
Splitting into 4 focused passes reduced LLM cognitive load per call. Each pass works within the 80K token limit while maintaining coherence via context chaining.

Deterministic + AI Hybrid Approach
Adding Tree-sitter AST parsing and static heuristics as a pre-pass reduced AI hallucination. The LLM now refines deterministic signals rather than generating architecture from scratch.

Graceful Fallback Strategy
When the multi-pass pipeline fails (rate limits, timeouts), automatic fallback to single-pass ensures users always receive a result. Quality degrades gracefully instead of failing entirely.

Quality Report & Confidence Scoring
Each analysis includes a coverage score, evidence density indicator, and low-confidence node flags so users can distinguish high-confidence from speculative outputs.

Planned Next Steps

High — **Expand Tree-sitter Language Support**
Add native AST parsing for Ruby, Rust, C#, and PHP — covering ~95% of repositories and eliminating regex fallback accuracy issues for these high-volume languages.

High — **Multi-Repo Microservice Analysis**
Allow linking multiple repositories as a single system. Cross-repo communication detection via API contract matching, shared schema detection, and service discovery patterns.

Medium — **Golden-Repo Evaluation Suite**
A curated benchmark of 20+ repos with expert-annotated ground-truth architecture maps and version-diff snapshots. Enables automated regression testing for architecture accuracy and version-diff correctness with each model update.

Medium — **Agent Layer for Evidence Verification**
A verification agent that cross-checks LLM-generated architecture claims against the deterministic AST signals, flags low-confidence nodes, and triggers guarded retries with more constrained prompts when confidence falls below a threshold.

Medium — **Observability & Model Health Monitoring**
Structured telemetry for latency per pass, failure modes by repo type, model drift detection (output quality change across Claude versions), and user trust metrics (re-analysis rates, correction frequency).

Comparison: CodeVision AI vs. Traditional Manual Review

DIMENSION	CODEVISION AI	MANUAL REVIEW
Time to Architecture Map	~45 seconds	Hours to days
Business Translation	Automated (founder & dev modes)	Requires separate stakeholder meeting
Technology Detection	44 signatures, ~90% precision	Expert-dependent, varies by familiarity
Consistency	Deterministic signals + reproducible LLM	Varies by reviewer and day
Edge Case Handling	Limited — known blind spots	Expert adapts to novel patterns
Deep Domain Knowledge	General-purpose LLM only	Domain experts available

DIMENSION	CODEVISION AI	MANUAL REVIEW
Runtime Behavior Analysis	Not supported (static analysis only)	Profiling, load testing, flame graphs